

INTRODUCTION TO PROGRAMMING AND CS USING PYTHON

TYPES OF KNOWLEDGE

- Declarative knowledge - is statements of facts
- Imperative knowledge is a recipe or "how to"
- Programming is about writing recipes to generate facts

Computers are the machines that execute algorithms
⇓

[In programming these recipes are algorithms

- Two things computers do-

→ Perform simple operation

100s of billions per second!

→ Remember results

100s of gigabytes of storage

What kind of calculations?

→ Once that you define

⇒ The Big Idea here?
⇓

A computer will only do what you tell it to do

ASPECTS OF LANGUAGES

- Primitive constructs

- English: words

- Programming language: numbers, strings, simple operators

- Syntan

- English - "cat dog boy" → not syntactically valid
 - "cat hugs boy" → syntactically valid

- Programming lang - "hi" 5 → not syntactically valid
 - "hi" * 5 → syntactically valid

- Static semantics: which syntactically valid strings have meaning

- English: "I am hungry" → syntactically valid

but static semantic error

- PL: "hi" + 5 → syntactically valid

but static semantic error

In english one sentence can have many meaning but
in programming programs have only one meaning



But the meaning may not be meaning
you intended



WHERE THINGS GO WRONG

- Syntatic errors
- Static semantic errors
- No linguistic errors but different meaning than what programmer intended
 - Program crashes, stops running
 - Program runs forever
 - Program gives an answer, but it's wrong

So what exactly we do when we write a program

↓↓
at the base of it
↓↓

we are going to create objects
and manipulate it ⇒ That's it

Objects have a type that defines the kinds of things
programs can do to them

↓
Scala

cannot be subdivided

→ Numbers - 1, 2, 3

→ Truth value - True, False

int
0, 1, 2

Float
0.21, 0.213

bool
True, False

none type
None } has only
one meaning

↓
Non-Scala

have internal structure

that can be accessed

→ list

→ dictionaries

→ sequence of characters
"abc"

TYPE CONVERSION (Casting)

- can convert object of one type to another
- `float(3)` casts the `int 3` to `float 3.0`

So now we know we create bunch of objects in program but what we do with them



we combine them with operators to form EXPRESSION



Python evaluates expression and stores the value. It doesn't store expressions!

- An expression has a value which has type.
 - `3+2` has value 5 and type `int`
- Syntax for a simple expression
`<object> <operator> <object>`

BIG IDEA

Replace complex expressions by one value

Work systematically to evaluate the expression

OPERATORS ON INT AND FLOAT

- $i + j \rightarrow$ the sum
 - $i - j \rightarrow$ the difference
 - $i * j \rightarrow$ the product
 - $i / j \rightarrow$ division $\longrightarrow$ result is always a float
- if both are ints, result is int
if either or both are floats, result is float
- $i // j \rightarrow$ floor division
 - $i \% j \rightarrow$ the remainder when i is divided by j
 - $i ** j \rightarrow i$ to the power of j

So many objects what we do with them?!



we assign names to them



and these names are called variables



In C $\Rightarrow$ It is bound to
one single value at a given time

BINDING VARIABLES TO VALUES

In CS, the equal sign is an **assignment**

- One value to one variable name
- Equal sign is not equality, not "solve for x"

An assignment binds a value to a name

variable = value

ABSTRACTING EXPRESSIONS

· Why give names to value of expressions?

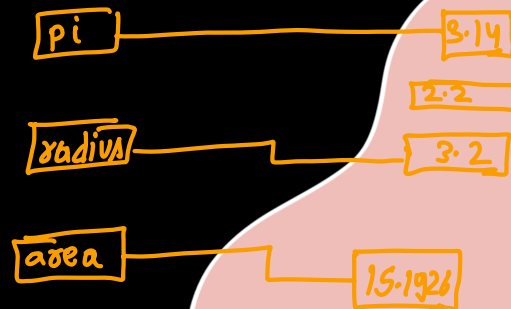
- To reuse names instead of values
- Makes code easier to read and modify

· Choose variable names wisely

- code needs to read
- Today, tomorrow, next year
- By you and others
- You'll be fine if you stick to letters, underscores, don't start with a number

CHANGE BINDINGS

- Can re-bind variable name using new assignment statement
- Previous value may still stored in memory but lost the handle for it.
- Value for `area` does not change until you tell the computer to do the calculation again



`pi = 3.14`

`radius = 2.2`

`area = pi * (radius ** 2)`

`radius = radius + 1`

BIG IDEA HERE



lines are evaluated one after the other

No skipping about, yet.

We'll see how lines can be skipped/repeated later